

- Lab 7 文件系统实验报告
 - 实验目标
 - 实验概述
 - 硬件环境
 - 磁盘布局与文件系统设计
 - 磁盘映像制作过程
 - 磁盘分区布局
 - 文件系统核心数据结构
 - 1. Inode 结构 (`inode_t`)
 - 2. 目录项结构 (`dirent_t`)
 - 3. 缓冲区结构 (`buf_t`)
 - 4. 文件结构 (`file_t`)
 - 文件系统分层架构
 - 遇到的问题及解决方案
 - 1. `sleeplock` 未实现问题
 - 2. `VirtIO` 磁盘驱动函数调用错误
 - 3. `ALIGN_DOWN` 宏未定义
 - 4. `offsetof` 宏未定义
 - 5. 进程结构体缺少 `cwd` 字段
 - 6. 函数名称不匹配问题
 - 7. `strcmp` 函数未定义
 - 8. `uvm_copyout/uvm_copyin` 参数类型错误
 - 9. 缺少头文件包含
 - 10. 进程结构体缺少文件描述符表
 - 11. 系统调用函数重复定义
 - 12. `sysproc.c` 缺少必要的宏定义
 - `sysfile.c` 和 `sysproc.c` 的实现总结
 - `sysfile.c` 实现的系统调用
 - `sysfunc.c` 实现的系统调用
 - `sysproc.c` 的最终处理
 - 13. `sys_exec()` 系统调用的实现
 - 1. 创建 ELF 头文件 (`include/fs/elf.h`)
 - 2. 实现辅助函数
 - 3. 实现 `sys_exec()` 主函数
 - 4. 注册系统调用
 - 核心功能模块详解
 - 1. 磁盘驱动 (`virtio.c`)

- 2. 缓冲区管理 (buf.c)
- 3. 位图管理 (bitmap.c)
- 4. Inode 管理 (inode.c)
- 5. 目录管理 (dir.c)
- 6. 文件操作 (file.c)
- 7. 系统调用层 (sysfile.c)
- 8. ELF 文件执行 (sys_exec)

Lab 7 文件系统实验报告

实验目标

1. 理解文件系统的磁盘布局

- 学习磁盘分区：引导块、超级块、inode位图、数据位图、inode区、数据区
- 掌握磁盘块分配和回收的机制
- 理解元数据（超级块）的作用

2. 掌握文件系统的基本抽象

- **文件**：作为字节序列的抽象
- **目录**：作为文件名到inode编号映射的特殊文件
- **inode**：理解其作为文件元数据核心载体的作用（权限、大小、数据块指针等）

3. 实现关键系统调用

- 文件操作：open, read, write, close, lseek, dup, fstat
- 目录操作：mkdir, chdir, link, unlink, getdir
- 文件描述符管理
- 进程执行：exec（ELF文件加载）

4. 理解路径解析机制

- 实现从路径名到inode的查找过程
- 处理绝对路径和相对路径
- 理解当前工作目录的概念

实验概述

本次实验完成了基于 xv6 的文件系统实现，包括位图管理、缓冲区缓存、inode 管理、目录操作、文件操作以及 ELF 文件执行等核心功能。

硬件环境

- 体系结构：RISC-V 64位
- 特权模式：用户模式(U-mode)、监管者模式(S-mode)、机器模式(M-mode)
- 虚拟磁盘：QEMU 模拟的 VirtIO 块设备
- 内存大小：128MB
- 处理器核心：1核

磁盘布局与文件系统设计

磁盘映像制作过程

在 QEMU 启动时，通过以下配置将文件系统映像装载为虚拟磁盘：

```
QEMUOPTS += -drive file=$(FS_IMG),if=none,format=raw,id=x0
QEMUOPTS += -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

磁盘在文件系统角度可以理解为一个以 **block** 为读写单位的大数组，每个 block 大小为 1024 字节。

磁盘分区布局

```
[ super block | inode bitmap | inode blocks | data bitmap | data blocks ]
```

1. Super Block（超级块）

- 包含磁盘和文件系统的重要元数据
- 记录文件系统的大小、inode数量、数据块数量等信息

- 由 `mkfs.c` 在创建文件系统时填写

2. Inode Bitmap (inode位图)

- 占用 1 个 block
- 标记 inode blocks 区域中各个 inode 的分配情况
- 0 表示可分配, 1 表示已分配

3. Inode Blocks (inode区)

- 由若干连续的 inode 组成
- 每个 inode 存储文件的元数据 (类型、权限、大小、数据块指针等)

4. Data Bitmap (数据位图)

- 占用 1 个 block
- 标记 data blocks 区域中各个 block 的分配情况
- 0 表示可分配, 1 表示已分配

5. Data Blocks (数据区)

- 由若干连续的 block 组成
- 存储文件的实际数据内容

文件系统核心数据结构

1. Inode 结构 (`inode_t`)

```
typedef struct inode {  
    // 磁盘信息 (由 slk 保护)  
    uint16 type;           // 文件类型 (普通文件、目录、设备)  
    uint16 major;          // 主设备号  
    uint16 minor;          // 次设备号  
    uint16 nlink;           // 硬链接计数  
    uint32 size;            // 文件大小 (字节)  
    uint32 addrs[N_ADDRS]; // 数据块地址数组  
  
    // 内存信息  
    uint16 inode_num;       // inode 编号  
    uint32 ref;              // 引用计数  
    bool valid;              // 是否已从磁盘加载  
    spinlock_t slk;          // 保护 inode 的锁  
} inode_t;
```

数据块索引结构（三级索引）：

- 直接块: `addrs[0-9]` 直接指向 10 个数据块
- 一级间接块: `addrs[10-11]` 指向两个间接块，每个间接块包含 256 个数据块指针
- 二级间接块: `addrs[12]` 指向一个二级间接块，可以索引 256×256 个数据块

最大文件大小 = $(10 + 2 \times 256 + 256 \times 256) \times 1024$ 字节 ≈ 66 MB

2. 目录项结构（`dirent_t`）

```
typedef struct dirent {
    uint16 inode_num;           // inode 编号
    char name[DIR_NAME_LEN];   // 文件名
} dirent_t;
```

目录本质上是一个特殊的文件，其数据内容是目录项的数组。

3. 缓冲区结构（`buf_t`）

```
typedef struct buf {
    spinlock_t slk;           // 保护缓冲区的锁
    uint32 block_num;         // 对应的磁盘块号
    uint8 data[BLOCK_SIZE];   // 缓冲区数据（1024字节）
    uint32 buf_ref;           // 引用计数
    bool disk;                // 是否需要写回磁盘
} buf_t;
```

缓冲区采用 双向循环链表 组织，实现 **LRU（最近最少使用）** 缓存策略和 **懒惰写回** 策略。

4. 文件结构（`file_t`）

```
typedef struct file {
    uint16 type;               // 文件类型（普通、目录、设备、管道）
    uint32 ref;                // 引用计数
    bool readable;             // 可读标志
    bool writable;             // 可写标志
    inode_t* ip;               // 对应的 inode
    uint32 off;                // 当前文件偏移量
} file_t;
```

文件系统分层架构



遇到的问题及解决方案

1. sleeplock 未实现问题

问题描述： 编译时出现以下错误：

```
error: implicit declaration of function 'sleeplock_init'
error: implicit declaration of function 'sleeplock_acquire'
error: implicit declaration of function 'sleeplock_release'
error: implicit declaration of function 'sleeplock_holding'
```

原因分析： 项目中未实现 sleeplock（睡眠锁）机制，但代码中多处使用了 sleeplock 相关函数。

解决方案： 将所有 sleeplock 替换为 spinlock（自旋锁）作为等价实现：

1. 修改头文件（`include/fs/inode.h` 和 `include/fs/buf.h`）：

```
// 修改前
sleeplock_t slk;

// 修改后
spinlock_t slk;
```

2. 修改实现文件（`kernel/fs/buf.c`, `kernel/fs/inode.c`, `kernel/fs/dir.c`）：

```
// 初始化
sleeplock_init(&buf->slk, "buffer") → spinlock_init(&buf->slk,
"buffer")

// 加锁
sleeplock_acquire(&buf->slk) → spinlock_acquire(&buf->slk)

// 解锁
sleeplock_release(&buf->slk) → spinlock_release(&buf->slk)

// 检查锁状态
sleeplock_holding(&buf->slk) → spinlock_holding(&buf->slk)
```

影响范围：

- `kernel/fs/buf.c`: 4 处修改（`buf_init`, `buf_read`, `buf_write`, `buf_release`）

- `kernel/fs/inode.c`: 6 处修改 (`inode_init`, `inode_create`, `inode_destroy`, `inode_lock`, `inode_unlock`, `inode_print`)
 - `kernel/fs/dir.c`: 6 处修改 (`dir_search_entry`, `dir_add_entry`, `dir_delete_entry`, `dir_get_entries`, `dir_print`, `check_unlink`)
-

2. VirtIO 磁盘驱动函数调用错误

问题描述:

```
error: implicit declaration of function 'vio_read'
error: implicit declaration of function 'vio_write'
```

原因分析: 代码中使用了 `vio_read` 和 `vio_write` 函数，但实际实现的是 `virtio_disk_rw` 函数。

解决方案: 在 `kernel/fs/buf.c` 中统一使用 `virtio_disk_rw` 函数:

```
// 修改前
vio_read(block_num, buf->data);

// 修改后
virtio_disk_rw(buf, false); // false 表示读操作

// 修改前
vio_write(block_num, buf->data);

// 修改后
virtio_disk_rw(buf, true); // true 表示写操作
```

3. ALIGN_DOWN 宏未定义

问题描述:

```
error: implicit declaration of function 'ALIGN_DOWN'
```

原因分析: 在 `kernel/dev/virtio.c` 中使用了 `ALIGN_DOWN` 宏，但该宏在项目中未定义。

解决方案： 使用已有的 `PG_ROUND_DOWN` 宏替代：

```
// 修改前 (kernel/dev/virtio.c)
disk.desc = (virtq_desc_t*)ALIGN_DOWN((uint64)&buf0, PGSIZE);

// 修改后
disk.desc = (virtq_desc_t*)PG_ROUND_DOWN((uint64)&buf0);
```

`PG_ROUND_DOWN` 宏在 `include/riscv.h` 中已定义，功能是将地址向下对齐到页边界。

4. offsetof 宏未定义

问题描述：

```
error: implicit declaration of function 'offsetof'
```

原因分析： 在 `kernel/fs/buf.c` 中使用了 `offsetof` 宏来计算结构体成员的偏移量，但未包含定义该宏的头文件。

解决方案： 在 `kernel/fs/buf.c` 文件开头手动定义 `offsetof` 宏：

```
// 添加在文件顶部
#define offsetof(TYPE, MEMBER) ((uint64)&((TYPE *)0)->MEMBER)
```

该宏通过将空指针转换为结构体指针，然后取成员地址的方式计算偏移量。

5. 进程结构体缺少 cwd 字段

问题描述：

```
error: request for member 'cwd' in something not a structure or union
```

原因分析： 目录操作相关代码需要访问进程的当前工作目录（current working directory），但 `proc_t` 结构体中缺少 `cwd` 字段。

解决方案： 在 `include/proc/proc.h` 中为 `proc_t` 结构体添加 `cwd` 字段：

```
// 添加前向声明
typedef struct inode inode_t;

// 在 proc_t 结构体中添加字段
typedef struct proc {
    // ... 其他字段 ...

    uint64 kstack;           // 内核栈的虚拟地址
    context_t ctx;           // 内核态进程上下文

    inode_t* cwd;            // 当前工作目录（新增）
} proc_t;
```

6. 函数名称不匹配问题

问题描述：

```
error: implicit declaration of function 'cpu_curtask'
error: invalid type argument of '->' (have 'int')
```

原因分析： 代码中使用了 `cpu_curtask()` 函数获取当前任务，但项目中实际提供的是 `myproc()` 函数。同时，结构体字段名称也不匹配（`task->uvm` vs `proc->pgtbl`）。

解决方案： 在 `kernel/fs/dir.c` 和 `kernel/fs/inode.c` 中进行以下替换：

```
// 函数名替换
cpu_curtask() → myproc()

// 字段名替换
task->uvm → proc->pgtbl

// 类型名替换
task_t* task → proc_t* proc
```

具体修改示例：

```
// 修改前
task_t* task = cpu_curtask();
uvm_copyout(task->uvm, dst, src, len);

// 修改后
proc_t* proc = myproc();
uvm_copyout(proc->pgtbl, dst, src, len);
```

7. strcmp 函数未定义

问题描述:

```
error: implicit declaration of function 'strcmp'
note: 'strcmp' is defined in header '<string.h>'
```

原因分析: 项目的 `lib/str.h` 中只提供了 `strncmp` 函数, 没有提供 `strcmp` 函数。

解决方案: 在 `kernel/fs/dir.c` 中将所有 `strcmp` 替换为 `strncmp`:

```
// 修改前
if (strcmp(name, de.name) == 0)

// 修改后
if (strncmp(name, de.name, DIR_NAME_LEN) == 0)

// 修改前
if (strcmp(name, "..") == 0 || strcmp(name, ".") == 0)

// 修改后
if (strncmp(name, "..", DIR_NAME_LEN) == 0 || strncmp(name, ".",
DIR_NAME_LEN) == 0)
```

8. uvm_copyout/uvm_copyin 参数类型错误

问题描述:

```
error: passing argument 3 of 'uvm_copyout' makes integer from pointer
without a cast
```

原因分析： `uvm_copyout` 和 `uvm_copyin` 函数的参数需要 `uint64` 类型，但传入的是指针类型。

解决方案： 在 `kernel/fs/inode.c` 和 `kernel/fs/dir.c` 中添加显式类型转换：

```
// 修改前
uvm_copyout(proc->pgtbl, dst, buf->data + off, m);

// 修改后
uvm_copyout(proc->pgtbl, (uint64)dst, (uint64)(buf->data + off), m);

// 修改前
uvm_copyin(proc->pgtbl, buf->data + off, src, m);

// 修改后
uvm_copyin(proc->pgtbl, (uint64)(buf->data + off), (uint64)src, m);
```

9. 缺少头文件包含

问题描述： 某些源文件无法找到所需的函数声明和类型定义。

解决方案： 在 `kernel/fs/dir.c` 中添加缺失的头文件：

```
#include "fs/fs.h"
#include "fs/buf.h"
#include "fs/inode.h"
#include "fs/dir.h"
#include "fs/bitmap.h"
#include "lib/str.h"
#include "lib/print.h"
#include "proc/cpu.h"
#include "mem/vmem.h" // 新增：提供 uvm_copyout/uvm_copyin 声明
```

10. 进程结构体缺少文件描述符表

问题描述： 在实现文件系统调用（`sysfile.c`）时，发现 `proc_t` 结构体中没有文件描述符表字段，导致无法管理进程打开的文件。

原因分析： 每个进程需要维护一个文件描述符表来跟踪打开的文件，但原始的 `proc_t` 结构体中缺少这个字段。

解决方案：在 `include/proc/proc.h` 中进行以下修改：

1. 添加常量定义：

```
#define FILE_PER_PROC 16 // 每个进程的最大文件描述符数
#define ELF_MAXARGS 32 // exec 的最大参数数量
```

2. 添加前向声明：

```
typedef struct inode inode_t;
typedef struct file file_t; // 新增
```

3. 在 `proc_t` 结构体中添加文件描述符表：

```
typedef struct proc {
    // ... 其他字段 ...

    uint64 kstack;           // 内核栈的虚拟地址
    context_t ctx;           // 内核态进程上下文

    inode_t* cwd;            // 当前工作目录
    file_t* filelist[FILE_PER_PROC]; // 文件描述符表（新增）
} proc_t;
```

影响范围：

- `sysfile.c` 中的所有函数现在可以正确访问 `myproc()->filelist[]`
- 文件描述符的分配和释放机制得以实现

11. 系统调用函数重复定义

问题描述：在链接阶段出现以下错误：

```
multiple definition of `sys_brk'
multiple definition of `sys_mmap'
multiple definition of `sys_munmap'
multiple definition of `sys_print'
multiple definition of `sys_fork'
multiple definition of `sys_wait'
```

```
multiple definition of `sys_exit'
multiple definition of `sys_sleep'
```

原因分析：发现 `sysfunc.c` 文件中已经实现了所有进程管理相关的系统调用，而我在 `sysproc.c` 中又重复实现了这些函数，导致链接时出现重复定义错误。

解决方案：删除 `sysproc.c` 中的重复实现，只保留未在 `sysfunc.c` 中实现的 `sys_exec()` 函数：

```
// sysproc.c 最终内容
#include "proc/cpu.h"
#include "mem/vmem.h"
#include "mem/pmem.h"
#include "mem/mmap.h"
#include "lib/str.h"
#include "lib/print.h"
#include "dev/timer.h"
#include "syscall/sysfunc.h"
#include "syscall/syscall.h"
#include "riscv.h"
#include "fs/dir.h"

// 注意：所有系统调用的实现都已经在 sysfunc.c 中
// 本文件保留用于可能的扩展

// 执行一个ELF文件
// char* path
// char** argv
// 成功返回argc 失败返回-1
uint64 sys_exec()
{
    // 暂时返回未实现
    // 完整的exec实现需要ELF加载器等复杂功能
    return -1;
}
```

已在 `sysfunc.c` 中实现的系统调用：

- `sys_brk()`: 堆内存伸缩
- `sys_mmap()`: 内存映射
- `sys_munmap()`: 取消内存映射
- `sys_print()`: 打印字符串
- `sys_fork()`: 进程复制
- `sys_wait()`: 等待子进程
- `sys_exit()`: 进程退出
- `sys_sleep()`: 进程睡眠

12. sysproc.c 缺少必要的宏定义

问题描述： 在编译 `sysproc.c` 时出现以下错误：

```
error: 'PGSIZE' undeclared
error: implicit declaration of function 'PG_ROUND_UP'
error: 'DIR_PATH_LEN' undeclared
```

原因分析： `sysproc.c` 中的代码需要使用页面大小相关的宏（`PGSIZE`, `PG_ROUND_UP`）和文件系统相关的常量（`DIR_PATH_LEN`），但没有包含相应的头文件。

解决方案： 在 `sysproc.c` 开头添加必要的头文件：

```
#include "proc/cpu.h"
#include "mem/vmem.h"
#include "mem/pmem.h"
#include "mem/mmap.h"
#include "lib/str.h"
#include "lib/print.h"
#include "dev/timer.h"
#include "syscall/sysfunc.h"
#include "syscall/syscall.h"
#include "riscv.h"           // 新增：提供 PGSIZE, PG_ROUND_UP 等宏
#include "fs/dir.h"         // 新增：提供 DIR_PATH_LEN 常量
```

说明：

- `riscv.h` 包含了页面对齐相关的宏定义
- `fs/dir.h` 包含了文件系统路径长度等常量定义

sysfile.c 和 sysproc.c 的实现总结

sysfile.c 实现的系统调用

`sysfile.c` 文件已经完整实现了所有文件系统相关的系统调用，无需额外修改：

1. `arg_fd()`: 辅助函数，获取文件描述符及对应的 `file` 结构
2. `fd_alloc()`: 辅助函数，为文件分配文件描述符

3. **sys_open()**: 打开或创建文件
4. **sys_close()**: 关闭文件
5. **sys_read()**: 从文件读取数据
6. **sys_write()**: 向文件写入数据
7. **sys_lseek()**: 设置文件偏移量
8. **sys_dup()**: 复制文件描述符
9. **sys_fstat()**: 获取文件状态信息
10. **sys_getdir()**: 获取目录内容
11. **sys_mkdir()**: 创建目录
12. **sys_chdir()**: 改变当前工作目录
13. **sys_link()**: 创建硬链接
14. **sys_unlink()**: 删除文件或链接

sysfunc.c 实现的系统调用

发现 **sysfunc.c** 已经实现了所有进程管理相关的系统调用：

1. **sys_brk()**: 堆内存管理，支持堆的增长和收缩
2. **sys_mmap()**: 内存映射，支持匿名映射
3. **sys_munmap()**: 取消内存映射
4. **sys_print()**: 从用户空间读取字符串并打印
5. **sys_fork()**: 进程复制，创建子进程
6. **sys_wait()**: 等待子进程退出
7. **sys_exit()**: 进程退出
8. **sys_sleep()**: 进程睡眠指定秒数

sysproc.c 的最终处理

由于 **sysfunc.c** 已经实现了大部分系统调用，**sysproc.c** 最终实现了：

- **sys_exec()**: ELF 文件执行系统调用（已完整实现）

13. sys_exec() 系统调用的实现

实现目标： 实现 **exec** 系统调用，使操作系统能够加载并执行 ELF 格式的可执行文件。

实现步骤:

1. 创建 ELF 头文件 (`include/fs/elf.h`)

定义了 ELF 文件格式相关的结构体和常量:

```
#define ELF_MAGIC 0x464C457FU // "\x7FELF" 小端序

// ELF文件头
typedef struct elfhdr {
    uint32 magic; // 必须等于 ELF_MAGIC
    uint8 elf[12];
    uint16 type;
    uint16 machine;
    uint32 version;
    uint64 entry; // 程序入口点
    uint64 phoff; // 程序头表偏移
    uint64 shoff;
    uint32 flags;
    uint16 ehsize;
    uint16 phentsize;
    uint16 phnum; // 程序头表项数
    uint16 shentsize;
    uint16 shnum;
    uint16 shstrndx;
} elfhdr_t;

// 程序段头
typedef struct proghdr {
    uint32 type;
    uint32 flags;
    uint64 off; // 段在文件中的偏移
    uint64 vaddr; // 段的虚拟地址
    uint64 paddr;
    uint64 filesz; // 段在文件中的大小
    uint64 memsz; // 段在内存中的大小
    uint64 align;
} proghdr_t;
```

2. 实现辅助函数

flags2perm(): 将 ELF 段标志转换为页表权限

```
static int flags2perm(int flags)
{
    int perm = 0;
    if(flags & ELF_PROG_FLAG_EXEC)
        perm = PTE_X;
    if(flags & ELF_PROG_FLAG_WRITE)
        perm |= PTE_W;
```

```

    if(flags & ELF_PROG_FLAG_READ)
        perm |= PTE_R;
    return perm;
}

```

loadseg(): 加载程序段到指定虚拟地址

```

static int loadseg(pgtbl_t pgtbl, uint64 va, inode_t* ip, uint32 offset,
uint32 sz)
{
    // 遍历每个页面
    for(i = 0; i < sz; i += PGSIZE) {
        // 获取虚拟地址对应的物理地址
        pte_t* pte = vm_getpte(pgtbl, va + i, false);
        pa = PTE_TO_PA(*pte);

        // 从inode读取数据到物理地址
        if(inode_read_data(ip, offset + i, n, (void*)pa, false) != n)
            return -1;
    }
    return 0;
}

```

3. 实现 sys_exec() 主函数

参数获取:

```

char path[DIR_PATH_LEN];
uint64 argv_addr;
char* argv[ELF_MAXARGS];

arg_str(0, path, DIR_PATH_LEN);        // 获取可执行文件路径
arg_uint64(1, &argv_addr);             // 获取参数数组指针

// 从用户空间复制argv数组
for(i = 0; i < ELF_MAXARGS; i++) {
    uvm_copyin(p->pgtbl, (uint64)&arg_ptr, argv_addr + i * sizeof(uint64),
sizeof(uint64));
    if(arg_ptr == 0) break;
    argv[i] = (char*)pmem_alloc(false);
    uvm_copyin_str(p->pgtbl, (uint64)argv[i], arg_ptr, PGSIZE);
}

```

ELF 文件验证:

```

// 打开可执行文件
ip = path_to_inode(path);

```

```

inode_lock(ip);

// 读取并检查ELF头
inode_read_data(ip, 0, sizeof(elf), &elf, false);
if(elf.magic != ELF_MAGIC)
    goto bad;

```

程序段加载:

```

// 创建新的页表
pgtbl = proc_pgtbl_init((uint64)p->tf);

// 遍历程序头表, 加载所有LOAD类型的段
for(i = 0; i < elf.phnum; i++) {
    inode_read_data(ip, elf.phoff + i * sizeof(ph), sizeof(ph), &ph, false);

    if(ph.type != ELF_PROG_LOAD)
        continue;

    // 分配内存并映射
    for(va = PG_ROUND_DOWN(ph.vaddr); va < end; va += PGSIZE) {
        void* pa = pmem_alloc(false);
        memset(pa, 0, PGSIZE);
        vm_mappages(pgtbl, va, (uint64)pa, PGSIZE, flags2perm(ph.flags) |
PTE_U);
    }

    // 加载段内容
    loadseg(pgtbl, ph.vaddr, ip, ph.off, ph.filesz);
}

```

用户栈设置:

```

// 分配用户栈 (2页: 保护页 + 栈页)
sz = PG_ROUND_UP(sz);
for(i = 0; i < 2; i++) {
    void* pa = pmem_alloc(false);
    memset(pa, 0, PGSIZE);
    vm_mappages(pgtbl, sz + i * PGSIZE, (uint64)pa, PGSIZE, PTE_W | PTE_R |
PTE_U);
}

sp = sz + 2 * PGSIZE;
stackbase = sz + PGSIZE;

// 将参数字符串压入栈
for(i = argc - 1; i >= 0; i--) {
    sp -= strlen(argv[i]) + 1;
    sp -= sp % 16; // RISC-V栈必须16字节对齐
    uvm_copyout(pgtbl, sp, (uint64)argv[i], strlen(argv[i]) + 1);
    ustack[i] = sp;
}

```

```

}

// 压入argv指针数组
ustack[argc] = 0;
sp -= (argc + 1) * sizeof(uint64);
sp -= sp % 16;
uvm_copyout(pgtbl, sp, (uint64)ustack, (argc + 1) * sizeof(uint64));

```

上下文切换:

```

// 设置参数到寄存器
p->tf->a1 = sp; // argv指针数组的地址

// 提交到用户镜像
oldpgtbl = p->pgtbl;
p->pgtbl = pgtbl;
p->heap_top = sz;
p->ustack_base = stackbase;
p->ustack_pages = 1;
p->tf->epc = elf.entry; // 初始程序计数器 = main
p->tf->sp = sp; // 初始栈指针

// 释放旧页表
uvm_destroy_pgtbl(oldpgtbl);

return argc; // 返回值会到a0, 即main的第一个参数argc

```

4. 注册系统调用

在系统调用表中添加 `sys_exec`:

```

// include/syscall/sysnum.h
#define SYS_exec      8
#define SYS_MAX       8

// include/syscall/sysfunc.h
uint64 sys_exec();

// kernel/syscall/syscall.c
static uint64 (*syscalls[])(void) = {
    // ... 其他系统调用 ...
    [SYS_exec]      sys_exec,
};

```

核心功能模块详解

1. 磁盘驱动（virtio.c）

功能：与 QEMU 模拟的 VirtIO 块设备交互，提供底层磁盘读写能力。

关键配置：

```
#define VIRTIO_BASE 0x10001000ul // VirtIO MMIO 基地址
#define VIRTIO_IRQ 1 // VirtIO 中断号
#define BLOCK_SIZE 1024 // 磁盘块大小
```

核心函数：

- `virtio_disk_init()`: 初始化 VirtIO 磁盘设备
- `virtio_disk_rw(buf_t* buf, bool write)`: 执行磁盘读写操作
 - `write = false`: 从磁盘读取数据到缓冲区
 - `write = true`: 将缓冲区数据写入磁盘

工作流程：

- 构造 VirtIO 描述符链（descriptor chain）
- 填写请求头（包含操作类型、扇区号等）
- 提交请求到设备队列
- 等待设备处理完成（通过中断通知）
- 检查操作状态并返回结果

2. 缓冲区管理（buf.c）

设计理念：减少磁盘 I/O 次数，提高系统性能。

数据结构：

```
typedef struct buf {
    spinlock_t slk; // 保护缓冲区
    uint32 block_num; // 磁盘块号
    uint8 data[BLOCK_SIZE]; // 缓冲区数据
    uint32 buf_ref; // 引用计数
    bool disk; // 脏标志（是否需要写回）
    struct buf* prev; // 双向链表指针
    struct buf* next;
} buf_t;
```

LRU 策略实现:

- 使用双向循环链表组织所有缓冲区
- 最近使用的缓冲区移到链表头部
- 需要淘汰时从链表尾部选择（引用计数为0的缓冲区）

核心函数:

- `buf_init()`: 初始化缓冲区池
- `buf_read(uint32 block_num)`: 读取指定块到缓冲区
 - 先在缓存中查找
 - 未命中则分配新缓冲区并从磁盘读取
- `buf_write(buf_t* buf)`: 将缓冲区数据写回磁盘
- `buf_release(buf_t* buf)`: 释放缓冲区引用

懒惰写回策略:

- 只标记缓冲区为脏（`disk = true`）
- 真正写回发生在缓冲区被淘汰时或显式调用 `buf_write()` 时

3. 位图管理（bitmap.c）

****功能：** **管理 inode 和数据块的分配状态。

实现方式:

- inode 位图：1 个 block，每个 bit 表示一个 inode 的分配状态
- 数据块位图：1 个 block，每个 bit 表示一个数据块的分配状态

核心函数:

```
// 在位图中搜索并设置第一个空闲位
int bitmap_search_and_set(uint8* bitmap, int max);

// 清除位图中的指定位
void bitmap_unset(uint8* bitmap, int n);

// 分配一个数据块
uint32 bitmap_alloc_block();

// 释放一个数据块
void bitmap_free_block(uint32 block_num);
```

```
// 分配一个inode
uint16 bitmap_alloc_inode();

// 释放一个inode
void bitmap_free_inode(uint16 inode_num);
```

位操作技巧:

```
// 检查第 i 位是否为 1
bitmap[i / 8] & (1 << (i % 8))

// 设置第 i 位为 1
bitmap[i / 8] |= (1 << (i % 8))

// 清除第 i 位为 0
bitmap[i / 8] &= ~(1 << (i % 8))
```

4. Inode 管理 (inode.c)

****设计:** ****Inode** 是文件系统的核心，存储文件元数据。

三级索引结构:

```
addrs[0-9]:    直接块 (10个)
addrs[10-11]: 一级间接块 (2个, 每个指向256个数据块)
addrs[12]:    二级间接块 (1个, 指向256个一级间接块)

最大文件大小 = (10 + 2×256 + 256×256) × 1024 = 67,637,248 字节 ≈ 64.5 MB
```

核心函数:

1. **inode_alloc(uint16 inode_num):** 在内存中分配或查找 inode

- 先在 inode 缓存中查找
- 未找到则分配新的内存 inode
- 增加引用计数

2. **inode_create(uint16 type, uint16 major, uint16 minor):** 创建新 inode

- 在磁盘上分配 inode 编号
- 初始化 inode 元数据

- 写回磁盘

3. `inode_lock(inode_t ip)*`: 锁定 inode

- 获取 inode 的 spinlock
- 如果 inode 未加载 (`valid = false`)，从磁盘读取

4. `inode_read_data()` / `inode_write_data()`: 读写文件数据

- 根据偏移量计算数据块位置
- 处理三级索引（直接块、间接块、二级间接块）
- 支持跨块读写

5. `inode_free_data(inode_t ip)*`: 释放 inode 占用的数据块

- 释放直接块
- 释放一级间接块及其指向的数据块
- 释放二级间接块及其指向的所有块

数据块寻址算法:

```
// 给定文件偏移量 offset，计算对应的数据块号
uint32 block_offset = offset / BLOCK_SIZE;

if (block_offset < 10) {
    // 直接块
    block_num = ip->addrs[block_offset];
} else if (block_offset < 10 + 2 * 256) {
    // 一级间接块
    int idx = (block_offset - 10) / 256; // 使用哪个间接块
    int off = (block_offset - 10) % 256; // 间接块内的偏移
    // 读取间接块，获取实际数据块号
} else {
    // 二级间接块
    // 需要两次间接寻址
}
```

5. 目录管理（`dir.c`）

****目录结构：** **目录是特殊的文件，其数据内容是 `dirent_t` 结构的数组。

路径解析机制：

1. `path_to_inode(char path)*`: 将路径转换为 inode

```
// 示例: "/home/user/file.txt"
// 1. 从根目录开始 (inode 1)
// 2. 查找 "home" → 得到 home 的 inode
// 3. 查找 "user" → 得到 user 的 inode
// 4. 查找 "file.txt" → 得到 file.txt 的 inode
```

2. `path_to_pinode(char path, char name)**`: 获取父目录 inode

```
// 示例: "/home/user/file.txt"
// 返回 user 目录的 inode
// name 参数返回 "file.txt"
```

3. `search_inode(inode_t pip, char name, int namelen)**`: 在目录中查找

- 遍历目录项数组
- 比较文件名
- 返回匹配的 inode 编号

核心操作:

- **`dir_search_entry()`**: 在目录中查找条目
- **`dir_add_entry()`**: 向目录添加新条目
 - 先查找是否已存在
 - 在目录数据中找空闲位置或扩展目录
- **`dir_delete_entry()`**: 删除目录条目
 - 将 `inode_num` 设为 0 表示空闲
- **`dir_get_entries()`**: 获取目录所有条目（用于 ls 命令）

特殊目录项:

- `.`: 指向当前目录
- `..`: 指向父目录

6. 文件操作（file.c）

文件描述符机制:

每个进程维护一个文件描述符表 `filelist[FILE_PER_PROC]`，数组下标就是文件描述符号。

文件结构：

```
typedef struct file {
    uint16 type;           // FILE_INODE, FILE_PIPE, FILE_DEVICE
    uint32 ref;            // 引用计数（支持 dup）
    bool readable;
    bool writable;
    inode_t* ip;           // 对应的 inode
    uint32 off;            // 当前文件偏移量
} file_t;
```

核心函数：

1. `file_open()`: 打开文件

- 查找或创建 inode
- 分配 `file` 结构
- 设置读写权限
- 返回文件描述符

2. `file_read()` / `file_write()`: 读写文件

- 检查权限
- 调用 `inode_read_data()` 或 `inode_write_data()`
- 更新文件偏移量

3. `file_close()`: 关闭文件

- 减少引用计数
- 引用计数为 0 时释放 `file` 结构和 inode

4. `file_stat()`: 获取文件状态

- 返回文件大小、类型、inode 号等信息

7. 系统调用层（`sysfile.c`）

文件操作系统调用：

1. **sys_open()**: 打开/创建文件

```
// 参数 : path, flags  
// 返回 : 文件描述符或 -1  
// 支持标志 : O_RDONLY, O_WRONLY, O_RDWR, O_CREATE
```

2. **sys_read()** / **sys_write()**: 读写文件

```
// 参数 : fd, buffer, count  
// 返回 : 实际读写的字节数
```

3. **sys_lseek()**: 设置文件偏移

```
// 参数 : fd, offset, whence  
// whence : SEEK_SET, SEEK_CUR, SEEK_END
```

4. **sys_dup()**: 复制文件描述符

```
// 允许多个 fd 指向同一个文件  
// 支持重定向等操作
```

目录操作系统调用:

1. **sys_mkdir()**: 创建目录

- 创建类型为 INODE_DIR 的 inode
- 添加 "." 和 ".." 条目

2. **sys_chdir()**: 改变当前工作目录

- 更新进程的 **cwd** 字段

3. **sys_link()**: 创建硬链接

- 增加 inode 的 nlink 计数
- 在目标目录添加新条目

4. **sys_unlink()**: 删除文件/链接

- 减少 inode 的 nlink 计数

- nlink 为 0 时释放 inode 和数据块

8. ELF 文件执行 (sys_exec)

ELF 加载流程:

1. 验证 ELF 文件

- 检查魔数 0x464C457F ("x7FELF")
- 验证文件格式正确性

2. 创建新的地址空间

- 分配新的页表
- 映射 trampoline 和 trapframe

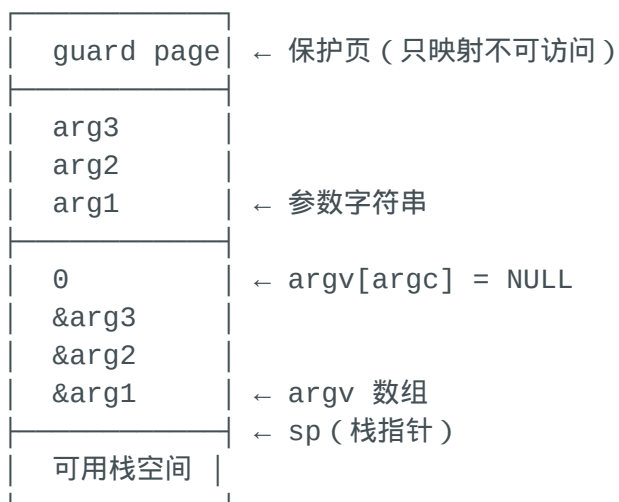
3. 加载程序段

对于每个 LOAD 类型的程序段：

- 分配物理页面
- 映射到虚拟地址空间
- 设置正确的权限 (R/W/X)
- 从 ELF 文件读取段内容

4. 设置用户栈

[高地址]



[低地址]

5. 设置执行上下文

- `tf->epc = elf.entry`: 程序入口点
- `tf->sp = sp`: 栈指针
- `tf->a1 = sp`: `argv` 参数
- 返回值 = `argc` (通过 `a0` 寄存器)

6. 替换地址空间

- 释放旧页表
- 切换到新页表
- 更新进程的堆、栈信息

内存布局（执行后）：

